SAE BUT2 MMI : Guide de Cadrage Technique

API REST, Git & Architecture Découplée

Département MMI

Février 2026

Pour ce projet, nous abandonnons le modèle "monolithique" au profit d'une architecture dite **découplée**. Le Front-end et le Back-end sont deux entités distinctes qui communiquent via des requêtes HTTP.

1. L'API RESTful

1.1 Principe

Votre serveur (Back-end) doit agir comme une API (Application Programming Interface). Il ne renvoie pas de HTML, mais des données structurées au format **JSON**.

— **Verbes HTTP :** Utilisez les méthodes standards (`GET` pour lire, `POST` pour créer, `PUT/PATCH` pour modifier, `DELETE` pour supprimer).

— **Points de terminaison (Endpoints) :** Vos URLs doivent être sémantiques. Exemple : `GET /api/sae` pour lister les SAE, `POST /api/login` pour l'authentification.

1.2 La communication entre le back et le front

1.2.1 Exemples de code de communication entre le front et le back

Avec Angular :

```
// src/app/services/sae.service.ts
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class SaeService {
  private apiUrl = 'http://localhost:8000/api/sae';

  constructor(private http: HttpClient) {}

  getListeSae() {
    // Si la route est protégée par JWT, il faudra ajouter le header :
    // const headers = new HttpHeaders().set('Authorization', `Bearer ${token}`);
    // return this.http.get(this.apiUrl, { headers });

    return this.http.get(this.apiUrl);
  }
}
```

Avec React :

```
// src/services/saeService.js
```

```
const API_URL = 'http://localhost:8000/api/sae';

export const saeService = {
  /**
   * Récupère la liste des SAE
   * @param {string|null} token - Le JWT si l'utilisateur est connecté
   */
  getListeSae: async (token = null) => {
    // 1. Préparation des options de la requête
    const options = {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json'
      }
    };

    // 2. Ajout du Token dans les headers si présent
    if (token) {
      options.headers['Authorization'] = `Bearer ${token}`;
    }

    // 3. Envoi de la requête
    const response = await fetch(API_URL, options);

    // 4. Vérification du statut (fetch ne lève pas d'erreur sur un 401 ou 500)
    if (!response.ok) {
      throw new Error(`Erreur HTTP : ${response.status}`);
    }

    // 5. Transformation du flux de données en objet JSON
    return await response.json();
  }
};
```

1.2.2 Gestion des problèmes CORS

Par défaut, le navigateur bloque les requêtes entre votre Front et votre Back (pas le même port). Sur Symfony, installez le bundle nécessaire :

```
composer require nelmio/cors-bundle
```

1.3 Exercice Pratique : Analyser et Manipuler une API REST

Avant de coder votre propre Back-end, vous allez explorer une API existante pour comprendre la structure des échanges de données.

1.3.1 Le format JSON, pour l'échange entre le back et le front

1. Ouvrez votre navigateur et accédez à l'URL suivante (API de test) :
   `https://jsonplaceholder.typicode.com/posts/1`
2. **Observation :** Ce que vous voyez est un objet **JSON**.
3. Identifiez les paires "clé / valeur". Quel est l'ID de l'article ? Quel est son titre ?

4. **Bilan :** Contrairement à un site web classique, l'API ne renvoie pas de balises `<div>` ou `<h1>`, mais uniquement de la donnée brute structurée. C'est votre Front-end (Angular) qui décidera comment l'afficher. Donc dans symfony, pas de `$this->render()` mais plutôt des `return new JsonResponse()` ou mieux, des `return $this->json()`,qui couplé avec le bundle Serializer (cf plus bas) permet d'envoyer simplement des données complexes et de les filtrer :

   ```
   composer require symfony/serializer-pack
   ```

   pour installer ce bundle, qui peut s'avérer utile. Il permettra de rajouter des annotations (`#[Groups(['sae:read'])]` par exemple) dans les entités pour filtrer les données renvoyées par groupe.

1.3.2 La Grammaire REST (Verbes HTTP)

Le principe de REST est d'utiliser les verbes HTTP pour définir l'action à réaliser sur une **Ressource** (ici, une SAE ou un article).

— **GET** `/api/sae` : Lister toutes les SAE.
— **GET** `/api/sae/42` : Récupérer uniquement la SAE ayant l'ID 42.
— **POST** `/api/sae` : Créer une nouvelle SAE (les données sont envoyées dans le corps de la requête).
— **PUT** `/api/sae/42` : Modifier entièrement la SAE n°42.
— **DELETE** `/api/sae/42` : Supprimer la SAE n°42.

1.3.3 Test avec un client API (Postman ou Thunder Client)

Pour tester une API sans coder de formulaire, on utilise un client spécialisé.

1. Installez l'extension **Thunder Client** dans VS Code (ou utilisez Postman).
2. Faites une requête `GET` sur `https://jsonplaceholder.typicode.com/posts`.
3. Faites une requête `POST` sur la même URL en envoyant un objet JSON dans le "Body" :

   ```
   {
     "title": "Nouvelle SAE MMI",
     "body": "Ceci est un test de création via API",
     "userId": 1
   }
   ```

4. Observez le **Code de statut HTTP** renvoyé par le serveur (ex : `201 Created`).

1.3.4 Avec symfony : API platform

```
composer require api
```

Cette API peut être très pratique si vous décidez d'utiliser symfony côté back : cela génère automatiquement les contrôleurs depuis les entités grâce à un simple attribut, et on peut gérer les différents rôles ! Voilà un exemple, qui permettra de créer des méthodes pour `GET /api/trucs, GET /api/trucs/2`, et des routes pour PUT, POST, et DELETE réservée aux admins....

```
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\Post;
```

```
use ApiPlatform\Metadata\Put;
use ApiPlatform\Metadata\Delete;

#[ApiResource(
    operations: [
        new Get(),
        new GetCollection(),
        new Post(security: "is_granted('ROLE_ADMIN')"),
        new Put(security: "is_granted('ROLE_ADMIN')"),
        new Delete(security: "is_granted('ROLE_ADMIN')")
    ]
)]
class Truc{....

}
```

1.3.5 Avec symfony : Serializer

Autre bundle qu'il sera utile d'installer : Serializer, qui permet de décider quels champs de chaque entité renvoyer ou non au front via `$this->json()` :

```
composer require symfony/serializer

use Symfony\Component\Serializer\Annotation\Groups;

class User
{
    #[Groups(['user:read'])]
    private int $id;

    #[Groups(['user:read'])]
    private string $email;

    private string $password; // X pas dans le groupe

    #[Groups(['user:read'])]
    private string $role;
}
```

et dans le contrôleur :

```
return $this->json($user,200,[],['groups' => ['user:read']]);
```

2. Authentification : Du Session-based au JWT

Dans le projet de SAE que nous vous proposons, vous pourrez vous authentifier soit par session (avec une entité User, une configuration comme vue dans la partie "authentification et sécurité" dans le TD symfony), soit grâce à un token JWT. Voilà une courte présentation de cette deuxième méthode.

2.1 Principe

Contrairement au PHP classique où le serveur stocke une session, une API est "stateless" (sans état). Nous utiliserons le **JWT (JSON Web Token)** :

1. L'utilisateur envoie ses identifiants au Back-end.
2. Le Back-end vérifie et renvoie un **Token** signé (une chaîne de caractères chiffrée).
3. Le Front-end (Angular, Vue, etc.) stocke ce token et l'envoie dans le **Header** de chaque requête suivante pour prouver l'identité de l'utilisateur.

2.2 Exercice Pratique : Comprendre le JWT (JSON Web Token)

L'objectif de cet exercice est de comprendre comment une API REST authentifie un utilisateur sans utiliser de sessions côté serveur.

2.2.1 Phase 1 : Observation et "Tampering"

1. Rendez-vous sur le site `https://jwt.io`.
2. Dans la section **encoder** puis **Payload**, remplacez le contenu par :

```
{
  "sub": "12345",
  "name": "Votre Nom",
  "role": "student"
}
```

3. Observez la chaîne codée à gauche. Elle se compose de trois parties séparées par des points : **Header.Payload.Signature**.
4. **Défi :** Modifiez, cette fois dans la section **decoder**, manuellement une seule lettre du jeton (Token) à gauche. Que se passe-t-il dans la section "Signature Verification" ?
5. *Conclusion :* Le serveur n'a pas besoin de base de données pour savoir que vous avez modifié votre rôle ; la signature ne correspond plus à la clé secrète.

2.2.2 Phase 2 : Le flux d'authentification dans la SAE

Dans votre projet, le flux devra respecter les étapes suivantes :

— **Login :** Le Front envoie les identifiants au Back (Symfony).
— **Émission :** Le Back vérifie, génère un JWT signé et le renvoie en JSON.
— **Stockage :** Le Front stocke ce JWT (souvent dans le `localStorage`).
— **Appels API :** Pour chaque requête (ex : récupérer la liste des SAE), le Front ajoute le token dans les headers HTTP : `Authorization: Bearer <token>`.

2.2.3 En pratique, côté back, avec symfony

L'utilisation de LexikJWTAuthenticationBundle est conseillée si vous souhaitez gérer l'authentification par jeton JWT.

```
composer require lexik/jwt-authentication-bundle
php bin/console lexik:jwt:generate-keypair
```

Il faudra alors configurer la route de login dans le routeur :

```
  api_login_check:
    path: /api/login_check
```

et modifier security.yaml pour dire que c'est ce bundle qui gère :

```
# config/packages/security.yaml
security:
    # 1. On définit l'encodeur de mots de passe
    password_hashers:
        App\Entity\User: 'auto'

    firewalls:
        # 2. Le pare-feu pour obtenir le token (sert uniquement à générer le token)
        login:
            pattern: ^/api/login
            stateless: true
            json_login:
                check_path: /api/login_check
                success_handler: lexik_jwt_authentication.handler.authentication_success
                failure_handler: lexik_jwt_authentication.handler.authentication_failure

        # 3. Le pare-feu pour protéger l'API
        api:
            pattern: ^/api
            stateless: true
            jwt: ~ # Active la vérification du token JWT sur toutes les routes /api
```

`POST http://localhost:8000/api/login_check` devrait alors renvoyer un objet JSON contenant le jeton.

3. Gestion de Version avec Git

L'usage de Git est obligatoire pour le travail en équipe. Il permet de fusionner vos codes sans écraser le travail des autres.

3.1 Workflow recommandé

— **Main/Master :** Version stable, prête pour le déploiement.
— **Develop/Feature branches :** Ne travaillez jamais directement sur la branche principale. Créez une branche par fonctionnalité (`git checkout -b feat-login`).
— **Commits :** Faites des messages clairs (ex : *"feat : ajout du filtre par semestre"*).

3.2 Exercice Pratique : Collaboration avec Git

L'objectif est de simuler le flux de travail réel d'une équipe de développement sans écraser le travail de vos collaborateurs.

3.2.1 Le serveur : GitLab

1. Créez-vous un compte sur `www.gitlab.com`.
2. **Sécurité (Important) :** Depuis 2021, GitLab n'accepte plus votre mot de passe habituel dans le terminal. Vous devez utiliser un **Token (PAT)** ou une **Clé SSH**. Pour ce TP, créez un "Personal Access Token" dans vos paramètres de profil (section Access Tokens) avec les droits "api" et "write_repository". Notez-le bien !
3. Créez un nouveau projet (Blank project) nommé `tp-git-votre-nom`.
4. Dans les paramètres du projet (*Project information > Members*), ajoutez votre binôme comme collaborateur avec le rôle "Maintainer".

3.2.2 Utilisation de Git en ligne de commande

1. **git config :** Avant tout, identifiez-vous auprès de git sur votre machine :
   `git config --global user.name "Votre Pseudo"`
   `git config --global user.email "votre@email.com"`
2. **git clone :** Récupérez votre projet GitLab sur votre ordinateur :
   `git clone https://gitlab.com/votre_pseudo/tp-git-votre-nom.git`
   (Utilisez votre Token si un mot de passe est demandé). Entrez dans le répertoire créé avec `cd`.
3. **Le fichier .gitignore :** On ne veut pas tout envoyer sur le serveur (fichiers temporaires, dossiers système). Créez un fichier nommé `.gitignore` à la racine. Écrivez dedans :
   `.DS_Store` (pour Mac)
   `node_modules/` (si vous faites du JS)
   `vendor/` (si vous faites du Symfony)
   Enregistrez. Désormais, Git ignorera ces éléments.
4. **git status & add :** Créez un nouveau fichier `test.txt`. Tapez `git status` pour voir que Git détecte un nouveau fichier "untracked". Indexez-le avec `git add test.txt` (ou `git add .` pour tout indexer).
5. **git commit :** Validez votre modification localement :
   `git commit -m "Mon premier commit : ajout de test.txt"`
6. **git push :** Envoyez vos changements sur GitLab :
   `git push origin main`. (L'ancienne branche *master* s'appelle désormais **main**). Vérifiez sur le site GitLab que votre fichier est bien là.
7. **git log :** Tapez `git log` pour voir l'historique de vos versions.
8. **Travailler avec le dépôt du binôme :** Rajoutez le dépôt de votre binôme :
   `git remote add binome [url_du_depot_du_binome]`.
   Faites `git remote -v` pour vérifier que vous voyez bien `origin` (le vôtre) et `binome`.

3.2.3 Synchronisation et Conflits

1. **git fetch & merge :** Si votre binôme a fait un `push`, vous ne l'avez pas encore.
   Faites `git fetch binome`. Cela récupère les infos mais ne modifie pas vos fichiers.
   Faites `git merge binome/main` pour fusionner ses changements dans votre dossier.
2. **git pull :** Il existe un raccourci qui fait le *fetch* et le *merge* en une seule commande : `git pull origin main`. C'est la commande que vous utiliserez le plus souvent pour mettre à jour votre travail.
3. **Gestion des conflits :** Modifiez tous les deux la **même ligne** du fichier `test.txt`. L'un de vous fait *add/commit/push*. Quand le deuxième essaie de faire son *push*, GitLab va refuser ! Il faut alors faire un `git pull`. Git va marquer le conflit dans le fichier avec des symboles «««< et »»»>. Ouvrez le fichier, choisissez la version finale, supprimez les symboles, puis faites *add/commit/push*.

3.2.4 Branches et Workflow moderne

1. **git checkout -b :** On ne travaille jamais directement sur *main*. Créez une branche :
   `git checkout -b ma-nouvelle-fonction`.
   Modifiez un fichier, faites un commit.
2. **push de branche :** Envoyez cette branche sur le serveur :
   `git push origin ma-nouvelle-fonction`.
3. **Merge Request (MR) :** Allez sur l'interface web de GitLab. Un bouton "Create Merge Request" apparaît. Cliquez dessus. C'est ici que votre binôme peut relire votre code et l'accepter pour qu'il rejoigne la branche *main*.
4. Une fois fusionnée, revenez sur *main* localement : `git checkout main` et supprimez votre branche devenue inutile : `git branch -d ma-nouvelle-fonction`.

4. Outils et Hébergement

4.1 Stack Technologique

- **Back-end :** Symfony (API Platform recommandé pour gagner du temps sur le CRUD) ou Node.js/Express.
- **Front-end :** Angular, Vue.js ou React (selon affinités du groupe).
- **Base de données :** MariaDB/MySQL via votre espace o2switch.

4.2 Déploiement (o2switch)

L'hébergement sur o2switch nécessite une configuration spécifique :

- Le **Back-end** doit être déployé dans un sous-domaine (ex : `api.votreprojet.fr`) ou un dossier spécifique.
- Le **Front-end** (build de production) sera placé à la racine ou via un routage spécifique.
- Attention à la configuration du fichier `.htaccess` pour gérer les routes Angular/Vue et éviter les erreurs 404 au rafraîchissement.